Project Manual

Project 7- Traffic Light Detection: Day and Night

Pratyaksha Jha 240150025
Somita Agarwal 240150035
Sunkari Sharanya 240150036
Neeharika 240150008

Overview

This project presents a computer vision system designed to automatically detect traffic lights in images and determine their color (Red, Yellow, or Green) in different light conditions and varying environments. The system integrates deep learning-based object detection with classical image processing techniques using OpenCV to get accurate results.

Primary objectives:

- Prepare a dataset containing labelled images of real-world traffic lights and train the YOLO model.
- Classify the detected traffic lights into red, yellow, or green using HSV color space.
- Display the results directly on the original image with bounding boxes and annotations.
- Test the model for different types of weather conditions and day and night.

System Architecture / Workflow

The system follows a sequential pipeline:

Input Image
↓
YOLO Model (Traffic Light Detection)
↓
Bounding Box Extraction
↓
Region Cropping
↓
HSV Color Space Conversion
↓
Pixel percentage calculation
↓
Final output with annotations

Methodology / Approach Used

1) Traffic Light Detection

We used a YOLO (You Only Look Once) model from the Ultralytics framework to detect traffic lights in images.

The model takes an image as input and returns bounding boxes around the detected traffic lights with a label below it along with the confidence scores.

2) Bounding Box Extraction

It also outputs the coordinates of the bounding boxes, which is used in further processing.

The confidence score tells us, “how confident or sure YOLO is that this is a traffic light.” For example: Conf = 0.92, it means that the YOLO is 92% confident it's a traffic light.

3) Region Cropping

The detected bounding box coordinates, which are obtained from the YOLO model, are used to extract the region containing the traffic light from the original image. This cropped region is used for detecting color using HSV color space.

4) Color Detection using HSV(Hue,Saturation,Value)

The image obtained by cropping out the bounding box is converted from BGR(Blue,Green,Red) to HSV color space. HSV is preferred because in HSV, colors are separated by hue,which makes it easier to filter them, and thus HSV color space works better at varying lighting conditions.

Color masks are created to detect pixels of that colour, for:

- Red (two ranges due to HSV wrap-around)
- Green
- Yellow

5) Pixel-Based Classification

The number of pixels for each color are calculated. The percentage of each color is calculated with respect to the total detected pixels.

6) Final Decision

The color with the highest percentage is selected as the final traffic light state.

Dataset Details

The dataset is taken from the Roboflow Universe. The model requires the dataset to be in a specific format with a data.yaml file and .txt file for each image describing the coordinates of the bounding box and the class it belongs to. So it contains 2 labels - traffic_light and no_traffic_light. It initially contained 1677 images. We performed data augmentation to introduce variability and to make the dataset align more with real data. We specifically performed the following steps - add a blur of 4.8px , increase/decrease the brightness by 50%, and add noise by 2.75 %. The final dataset contained 4031 images.

Here our dataset is split into 88% (3531 images) for training, 8%(336 images) for validation and 4%(164) for testing.

Implementation Details

1) We implemented all the codes of the project in Colab, specifically, the GPU T4 for training the model.
1) Install ‘ ultralytics’ and roboflow to import dataset from roboflow directly and to use the YOLO26 model.
2) Import YOLO and roboflow

```
!pip install ultralytics
!pip install roboflow
from ultralytics import YOLO
from roboflow import Roboflow
```

3) Import the dataset from Roboflow using API key

```
rf = Roboflow(api_key="APIKEY")
project =
rf.workspace("somitas-workspace").project("traffic-lights-ydbwt-3w828
")
dataset = project.version(3).download("yolo26")
```

4) Train the YOLO model:

   Here we are running on 100 epochs,with a standard image size of 640 pixels.

```
```

```
model = YOLO("yolo26n.pt")
train_results = model.train(
    data=dataset.location+"/data.yaml",
    epochs=100,
    imgsz=640,
    device=0,
)
```

6) The function analyse_traffic_lights takes in the image, along with the bounding box, and a confidence threshold. It returns the probability for each color, the final color of the traffic light is given as the one with the highest probability.

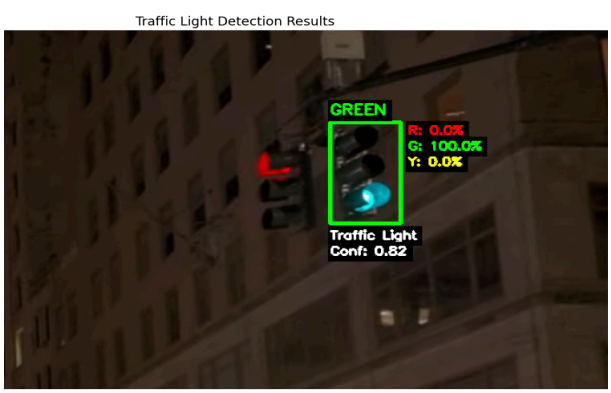
```
crop = img[y1:y2, x1:x2] #take only the bounding box from the entire image
hsv = cv2.cvtColor(crop, cv2.COLOR_BGR2HSV)  #use opencv to convert from bgr to hsv

```

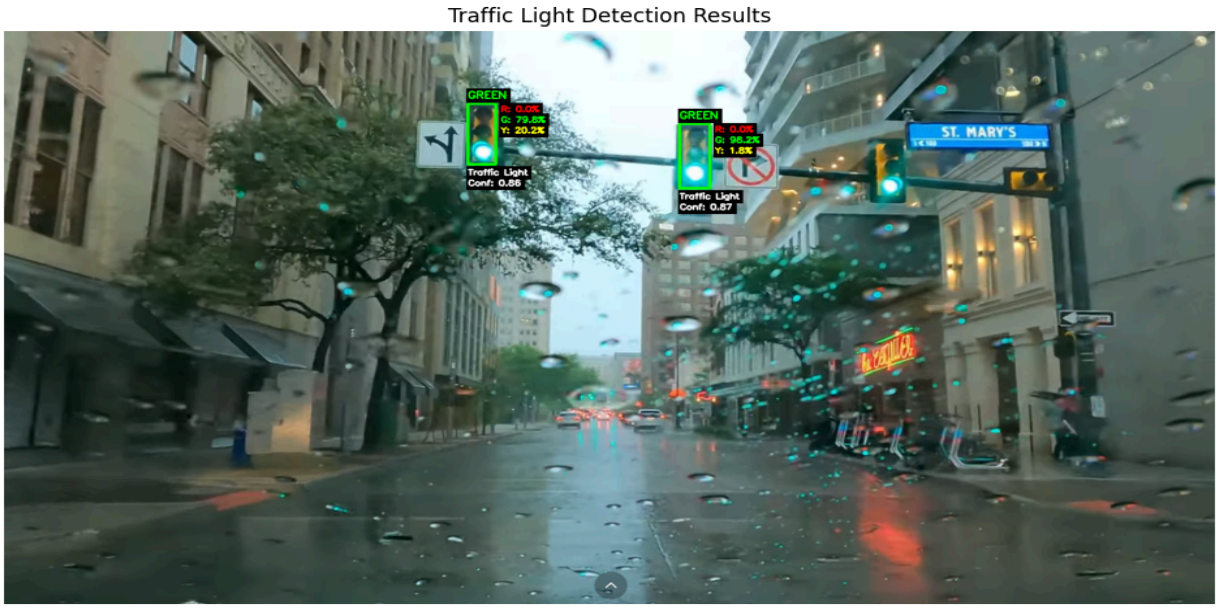
7) The function detect_lights() takes in the image path and draws a bounding box around the traffic lights and detects the color for that image.

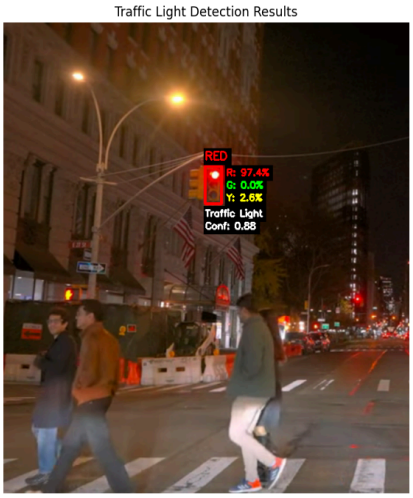
How to run

1) Import the already trained model attached with the zip file.
2) Run the function detect_lights() inside the colab notebook on a new image and provide the path to the image as an argument

Results and Discussion





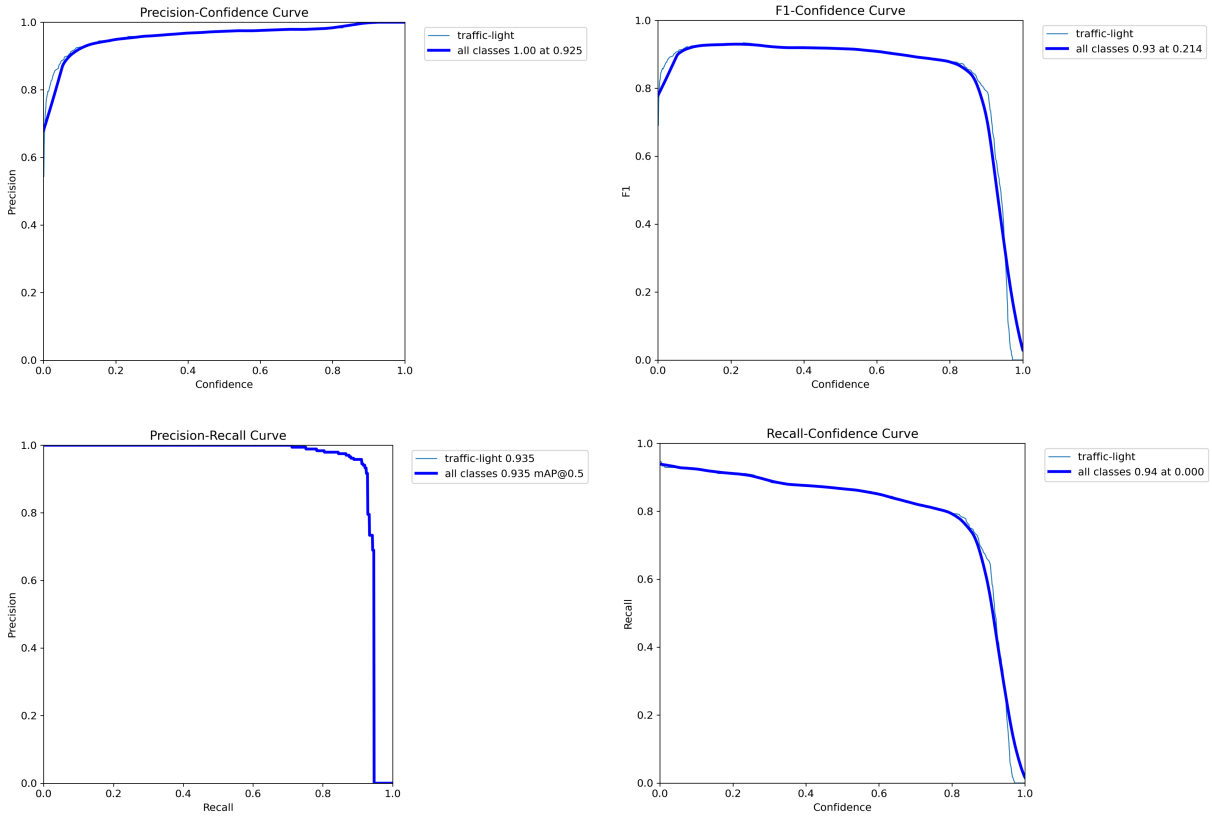


The above figures are the results of our model.The model accurately draws a bounding box around the traffic light with a probability for each color and a confidence score.

Precision	**96.7%**
Recall	**95.4%**
mAP50	**97.7%**
mAP50-95	**72.3%**

Hardware	**Tesla T4 GPU**
Inference speed	**3.7-4.7 ms per image**

These are the Precision-Confidence, F1-Confidence, Precision-Recall, and Recall-Confidence curves auto-generated by the model.val() function from the Ultralytics with fixed styling.



Limitations:

1) Results show that while the model can detect traffic lights accurately on rainy days or at night , it cannot accurately detect them in the combination of rainy nights because the colors seem to spread out.Because of this reason, it becomes difficult to identify the bounding box.
2) Traffic lights can only be detected up to a certain maximum distance. For example,if the traffic lights are too far away, the model does not work.

3) It is computationally heavy. Training on approximately 4000 images took more than 2 hours of time.
4) Our current model detects only vertical,front facing lights.

Future scope:

1) Performing some data preprocessing so that it becomes easier to detect traffic lights on rainy nights.
2) Expanding dataset to include horizontal and tilted traffic lights.
3) Use deep learning for colour classification.
4) Extend to real-time video processing.

References

1. Ultralytics YOLO GitHub
2. OpenCV Documentation
3. Roboflow Dataset Platform
4. Citations:

[1] N. H. Sarhan and A. Y. Al-Omary, "Traffic light Detection using OpenCV and YOLO," in *2022 International Conference on Innovation and Intelligence for Informatics, Computing, and Technologies (3ICT)*, 2022, doi: 10.1109/3ICT56508.2022.9990723.

[2] T. Sritarathorn and P. Pasupa, "Traffic Light Color Identification for Automatic Traffic Light Violation Detection System," in *2018 International Conference on Engineering, Applied Sciences, and Technology (ICEAST)*, Phuket, Thailand, 2018, pp. 1-4, doi: 10.1109/ICEAST.2018.8434400.

[3] V. Sharma, A. Gupta, and S. Kumar, "Traffic Light Detection and Recognition using YOLOv8," in *2024 3rd International Conference on Applied Artificial Intelligence and Computing (ICAAIC)*, Salem, India, 2024, pp. 1-6, doi: 10.1109/ICAAIC60222.2024.10575768.